

Saryu: a recurrent language model whose state is carried by Householder reflections

Technical report — draft v1 (architecture; comparisons pending)

Varun Sharma

15 September 2026

Abstract

Saryu is a recurrent language model with a fixed-size state. Every layer keeps one small vector per head, and every token moves that vector with two input-dependent Householder transforms $I - \beta \mathbf{u}\mathbf{u}^\top$ before a gate writes new content into it. The transform strength is parameterised as $\beta = 1 - \cos \theta$, so it lives in $[0, 2]$ by construction: $\beta = 2$ is an exact, norm-preserving reflection and a stationary point of the parameterisation, $\beta = 1$ a projection, $\beta < 1$ a contraction. Because reflections do not commute, the state depends on the *order* of the tokens, which no diagonal (commuting) transport can express; because every transform has spectral norm at most one and the write is convex, the state is provably bounded. The recurrence is trained with an exact chunk-parallel kernel (an inverse-free triangular solve per chunk) and generates at a per-token cost independent of context length. This draft describes the architecture, the kernel, the design reasoning and the two models trained so far (5M and 25M parameters, character-level enwik8, trained at context 128, and measured to use about 512 characters of context and no more). It reports only Saryu’s own measurements; matched comparisons against other architectures are the subject of the next version.

1 What Saryu is for

A transformer re-reads its whole context for every token it writes; its memory grows with the context. A recurrent model instead compresses the past into a state of fixed size and updates it once per token, so the cost of writing token t does not depend on t . The question for any recurrent design is what that fixed state can *represent*, and whether it can be trained in parallel.

Much recent work uses *diagonal* (element-wise) recurrences, where each coordinate of the state decays and is written independently [8]. Diagonal matrices commute, so the product of the transitions is the same whatever order the tokens came in. That is a real restriction: such models cannot track state through non-commuting operations without help [13, 15], and allowing negative eigenvalues in the transition is one way to recover it [7].

Saryu’s position is that order should be carried by the state itself, cheaply. Its transition is a product of Householder transforms [10]: rank-one corrections of the identity, each determined by one unit vector and one scalar. Two of them per token cost $O(d_h)$ per head, they can be exact reflections (norm-preserving, eigenvalue -1), and two reflections about different mirrors do not commute (Figure 4). The rest of the block is chosen so that this transport trains stably and in parallel.

Contributions of this draft.

1. A recurrent block whose per-head *vector* state is moved by input-dependent Householder transforms with $\beta = 1 - \cos \theta$, and written through a convex cosine gate (Section 2).
2. A boundedness guarantee for the state under any input (Proposition 1) and the commutativity argument for why the transport must not be diagonal (Section 3).
3. An exact chunk-parallel training kernel with a log-space rescaling that keeps it finite (Section 4).
4. Two trained character-level models and a description of what their transports learned (Section 5).

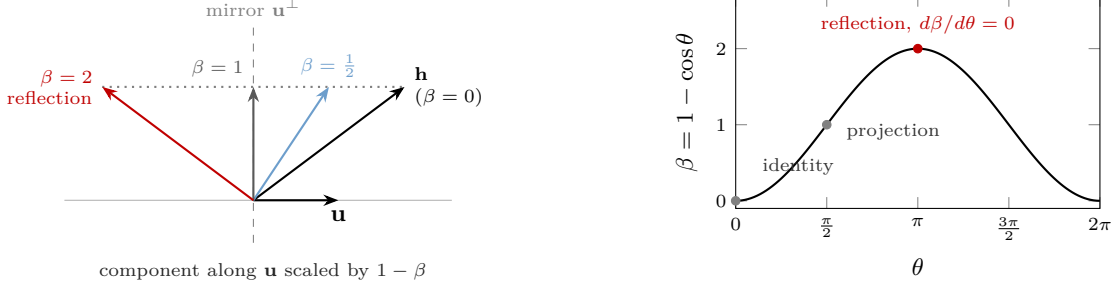


Figure 1: **Left:** one Householder transform $I - \beta \mathbf{u}\mathbf{u}^\top$ applied to a state $\mathbf{h}$. Only the component along $\mathbf{u}$ changes; at $\beta = 2$ the state is mirrored and its norm is unchanged. **Right:** the parameterisation $\beta = 1 - \cos \theta$ is bounded in $[0, 2]$ and has zero slope at the exact reflection $\theta = \pi$.

2 Architecture

2.1 The transport: one Householder transform

For a unit vector $\mathbf{u} \in \mathbb{R}^{d_h}$ and a scalar β , the generalised Householder transform is

$$H(\mathbf{u}, \beta) = I - \beta \mathbf{u}\mathbf{u}^\top. \quad (1)$$

It leaves every vector orthogonal to $\mathbf{u}$ unchanged and multiplies the component along $\mathbf{u}$ by $1 - \beta$. So β selects a regime (Figure 1, left): $\beta = 0$ is the identity, $\beta \in (0, 1)$ shrinks the component along $\mathbf{u}$, $\beta = 1$ deletes it (a projection), and $\beta = 2$ flips its sign — a reflection through the hyperplane $\mathbf{u}^\perp$, which is orthogonal and preserves the norm exactly.

Saryu parameterises the strength through an angle,

$$\beta = 1 - \cos \theta, \quad 1 - \beta = \cos \theta \in [-1, 1], \quad (2)$$

so the eigenvalue along $\mathbf{u}$ is literally $\cos \theta$. Three properties follow (Figure 1, right). The range $[0, 2]$ holds for every θ , so no input can push the transform outside it. The reflection $\beta = 2$ is reached *exactly*, at $\theta = \pi$ (a sigmoid $2\sigma(x)$ only approaches it). And $d\beta/d\theta = \sin \theta$ vanishes there, so a reflection is a stationary point that training does not drift away from by accident, while β can still move when the data asks for a contraction. The bias of the angle projection is initialised to π with zero weights, so every transform starts as an exact reflection.

2.2 One recurrent step

Each layer has $H = 8$ heads with state $\mathbf{h} \in \mathbb{R}^{d_h}$, $d_h = d/H$. For token t the layer computes, per head, $n_h = 2$ unit vectors $\mathbf{u}_{t,1}, \mathbf{u}_{t,2}$, their strengths $\beta_{t,1}, \beta_{t,2}$, a scalar gate g_t and a content vector $\mathbf{c}_t$, and updates

$$\mathbf{h}_t = a_t T_t \mathbf{h}_{t-1} + \mathbf{b}_t, \quad T_t = H(\mathbf{u}_{t,2}, \beta_{t,2}) H(\mathbf{u}_{t,1}, \beta_{t,1}), \quad a_t = 1 - g_t, \quad \mathbf{b}_t = g_t \mathbf{c}_t, \quad (3)$$

with the gate again an angle,

$$g_t = \min\left(\frac{1 - \cos \varphi_t}{2}, 0.9\right) \in [0, 0.9]. \quad (4)$$

The gate is a single scalar per head: a_t close to one keeps the transported past, g_t close to one replaces it with new content. The cap at 0.9 keeps $a_t \geq 0.1$, which the training kernel relies on (Section 4). Figure 2 draws the step.

2.3 The block and the model

The recurrence sits inside a gated block (Figure 3, right). The input is normalised with LayerNorm [3] and passed through a depthwise causal convolution of width 4, giving features $\mathbf{z}_t$ that see the last four tokens.

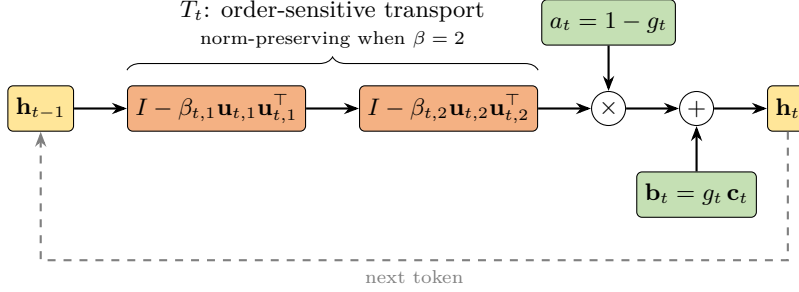


Figure 2: One recurrent step of one head (Eq. 3). All of $\mathbf{u}, \beta, g, \mathbf{c}$ are computed from the current token’s features, so the transport is input-dependent. The state is a vector of $d_h = d/8$ numbers.

Linear projections of $\mathbf{z}_t$ give the reflection vectors (normalised to unit length per head), the angles θ and φ , and the content $\mathbf{c}_t$. The per-head outputs of the recurrence are normalised with RMSNorm [22], multiplied by a SiLU [6] output gate computed from $\mathbf{z}_t$, and projected back to width d .

The language model (Figure 3, left) stacks N layers, each a Saryu block followed by a SwiGLU feed-forward block [16] of hidden width $2d$, both residual and pre-normalised. An optional *value embedding* — a second token embedding added to the content $\mathbf{c}_t$ of every block — is used in the 5M model. A final LayerNorm and a linear head give next-character logits.

A Saryu block has about $5d^2$ parameters (reflection vectors $2d^2$; content, output gate and output projections d^2 each) and a SwiGLU block $6d^2$.

2.4 Generation cost

At generation time a layer holds its H head states (d numbers in total) and the last three inputs of the convolution ($3d$ numbers). For the 25M model ($d = 744$, four layers) that is 11,904 numbers for the whole network, whether the context so far is ten characters or ten million, and each new token costs one pass through the layers. There is no cache that grows with the context.

3 Why the transport is built this way

3.1 Order: diagonal transports cannot see it

If every transition matrix commutes with every other — as diagonal or element-wise transitions do [8] — then a pure transport $\mathbf{h}_T = T_{x_T} \cdots T_{x_1} \mathbf{h}_0$ is the same for every ordering of the same tokens: the state is a function of the *multiset* of inputs. The best accuracy any such model can reach on a task is therefore bounded by the best function of the multiset, a number that can be computed exactly by enumeration. On the word problem of the symmetric group S_3 at length 8 that bound is 0.385 (chance is 0.167), while for the cyclic group $\mathbb{Z}_6$ of the same order it is 1.0, because there the multiset determines the answer.

In a small experiment with a width-64 transport, two commuting transports (a diagonal rotation and a single fixed Givens pairing) reached 0.388 ± 0.006 and 0.389 ± 0.003 on S_3 — on the bound — and 1.000 on $\mathbb{Z}_6$; a non-commuting transport (alternating Givens pairings) reached 0.779 ± 0.380 on S_3 over three seeds, above the bound that no commuting transport can cross.¹ Two reflections about different mirrors already fail to commute (Figure 4): in the plane they compose to rotations by $+90^\circ$ and -90° .

This is the same observation that motivates negative eigenvalues [7] and products of Householder transforms [17] in matrix-state linear recurrences. By the Cartan–Dieudonné theorem every orthogonal $d_h \times d_h$ matrix is a product of at most d_h reflections; with $n_h = 2$ per token a single step is a reflection pair (at $\beta = 2$, a rotation in one plane), and longer products are built up across tokens.

¹Repository `evidence/ceiling.py`, log `evidence/results/v2.txt`. The non-commuting arm there is a Givens butterfly, not Saryu’s reflections; the experiment establishes the ceiling, not Saryu’s accuracy.

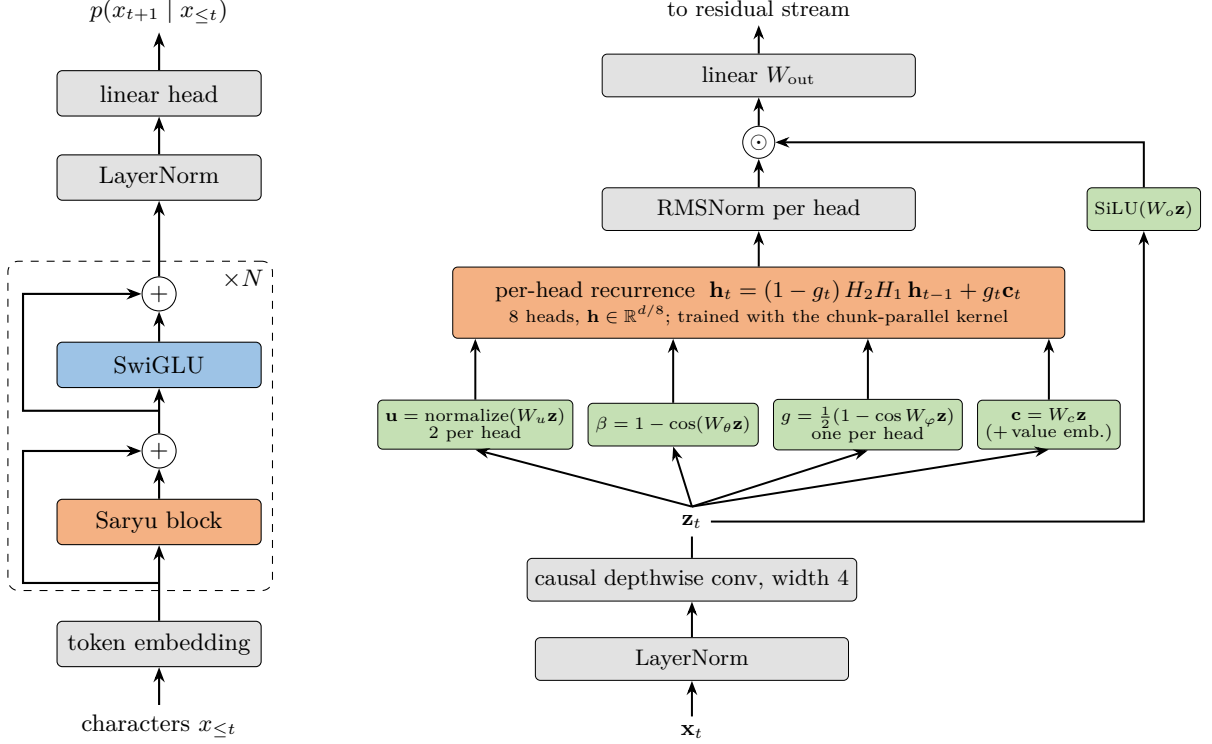


Figure 3: **Left:** the language model. Each of the N layers is a Saryu block and a SwiGLU block, both residual. The optional value embedding (a second token embedding) enters every Saryu block through its content $\mathbf{c}_t$. **Right:** the Saryu block. The convolution gives every projection a short local window; the recurrence carries everything older. At generation time each layer carries d recurrent numbers ($8 \text{ heads} \times d/8$) plus the convolution’s last three inputs.

3.2 Stability: the state cannot blow up

Proposition 1 *If $\beta \in [0, 2]$ for every transform and $g_t \in [0, 1]$, then for every input sequence*

$$\|\mathbf{h}_t\| \leq \max(\|\mathbf{h}_0\|, \max_{s \leq t} \|\mathbf{c}_s\|).$$

Proof. $H(\mathbf{u}, \beta)$ is symmetric with eigenvalues 1 (on $\mathbf{u}^\perp$) and $1 - \beta \in [-1, 1]$ (on $\mathbf{u}$), so its spectral norm is at most one, and so is that of T_t . Then $\|\mathbf{h}_t\| \leq (1 - g_t)\|\mathbf{h}_{t-1}\| + g_t\|\mathbf{c}_t\| \leq \max(\|\mathbf{h}_{t-1}\|, \|\mathbf{c}_t\|)$, and the claim follows by induction. $\square$

The bound is what the cosine parameterisation buys. An unconstrained β can leave $[0, 2]$ and make $|1 - \beta| > 1$, after which the state grows geometrically with length; in our early single-layer experiments an unbounded β produced state norms above 4×10^5 at length 4096. With Eq. 2 the bound holds whatever the weights are.

3.3 Why not a sigmoid, and why not a fixed reflection

A fixed $\beta = 2$ makes every step orthogonal but removes the model’s ability to forget along a direction, and it fixes $\det T_t = (-1)^{n_h}$ for every token. A sigmoid $\beta = 2\sigma(x)$ can approach a reflection but never reach it. In small state-tracking experiments on the groups S_4 and A_4 with $n_h \in \{2, 3\}$ (four seeds per cell, sixteen runs per parameterisation), the sigmoid solved 0 of 16 runs and settled near $\beta \approx 1$ (a projection); $1 - \cos \theta$ solved 9 of 16, reaching $\beta = 2.000$ where an exact reflection was consistent with the group and moving away from it

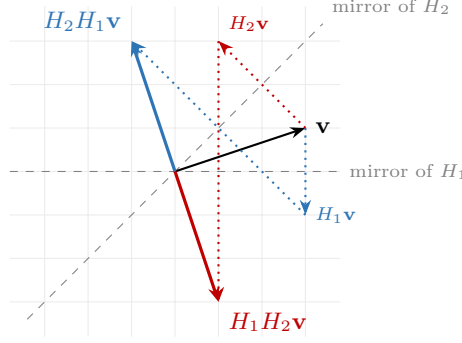


Figure 4: Order sensitivity. The same two reflections applied to the same state in opposite orders give different states (H_2H_1 is a rotation by $+90^\circ$, H_1H_2 by -90°). A recurrence built from them can tell “ a then b ” from “ b then a ” through the transport itself; a diagonal transport, whose factors commute, cannot.

where it was not.²

4 Training in parallel: the chunkwise kernel

Eq. 3 is sequential. Training it token by token would cost one Python-level step per token. Saryu instead processes chunks of $C = 8$ tokens at once, exactly, using the structure of Householder products [4]; the approach follows the chunkwise delta-rule kernels of [21], adapted to a vector state with an interleaved gated write.

Removing the gate. Within a chunk, let $\alpha_t = \prod_{k \leq t} a_k$ ($\alpha_0 = 1$) and rescale $\mathbf{s}_t = \mathbf{h}_t / \alpha_t$. Eq. 3 becomes $\mathbf{s}_t = T_t \mathbf{s}_{t-1} + \mathbf{e}_t$ with $\mathbf{e}_t = \mathbf{b}_t / \alpha_t$ and $\mathbf{e}_0 = \mathbf{h}_0$, the state carried in from the previous chunk. The gate has disappeared into the injections.

Unrolling the reflections. Number the $R = Cn_h$ transforms of the chunk $r = 1, \dots, R$ in the order they are applied, and let $\tau(r)$ be the token that transform r belongs to. Each transform subtracts $\beta_r y_r \mathbf{u}_r$ from the running vector, where y_r is $\mathbf{u}_r^\top$ of that vector just before transform r . The running vector at that moment is the sum of injections so far minus the earlier corrections, which gives

$$y_r + \sum_{r' < r} \beta_{r'} (\mathbf{u}_r^\top \mathbf{u}_{r'}) y_{r'} = \mathbf{u}_r^\top \sum_{j < \tau(r)} \mathbf{e}_j, \quad \text{i.e.} \quad (I + \text{tril}(G \text{diag}(\beta), -1)) \mathbf{y} = \mathbf{r}, \quad (5)$$

with Gram matrix $G_{rr'} = \mathbf{u}_r^\top \mathbf{u}_{r'}$. The system is unit lower triangular, so $\mathbf{y}$ comes from one triangular solve with no matrix inverse. All C outputs of the chunk then follow at once:

$$\mathbf{h}_t = \alpha_t \left(\sum_{j \leq t} \mathbf{e}_j - \sum_{r: \tau(r) \leq t} \beta_r y_r \mathbf{u}_r \right), \quad t = 1, \dots, C. \quad (6)$$

Keeping it finite. Computed naively, α_t is a product of gates and $1/\alpha_t$ in $\mathbf{e}_t$ overflows when a chunk contains several strong writes. Every physical quantity is a ratio $\alpha_t / \alpha_j \leq 1$, so the kernel works with $\log \alpha_t$ centred on its mean over the chunk. Each step lowers $\log \alpha_t$ by at most $\ln 10$ (the gate cap gives $a_t \geq 0.1$), so over $C = 8$ steps no value lies further than $\frac{1}{8} \sum_{k=1}^7 k \ln 10 \approx 8.1$ from the mean, and no rescaling factor exceeds $e^{8.1} \approx 3 \times 10^3$.

²Repository `evidence/beta_activation.py`. These are toy experiments with a single transport layer and are reported here only as the reason for the design choice.

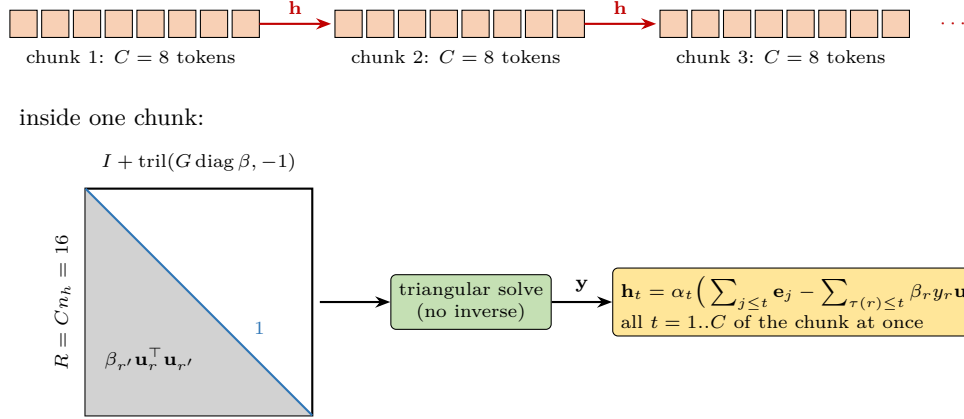


Figure 5: The chunkwise kernel. The sequence is cut into chunks of $C = 8$ tokens and the state is carried between chunks. Inside a chunk the 16 transforms couple only through a unit lower-triangular system (Eq. 5); one solve yields every output of the chunk (Eq. 6).

	Saryu 5M (checkpoint shipped with the code)	Saryu 25M
parameters	5,016,097	25,183,861
width d / layers / heads / n_h	320 / 4 / 8 / 2	744 / 4 / 8 / 2
head state d_h	40	93
value embedding	yes	no
training context / batch	128 / 48	128 / 48
steps (characters seen)	5,000 (30.7M)	10,000 (61.4M)
optimiser	AdamW, lr 10^{-3} , wd 0.01	AdamW, lr 10^{-3} , wd 0.01 (matrices)
schedule	one-cycle [18], 5% warm-up	warmup-stable-decay [19] (5% / 20%)
other	embeddings $10 \times$ lr; gate proj. $0.1 \times$ lr	z-loss 10^{-4} ; output proj. scaled $1/\sqrt{2N}$
gradient clipping	1.0	1.0
hardware, time per step	one T4 GPU, ≈ 0.2 s	one T4 GPU, 0.50 s

Table 1: The two trained models. Both use the same block (Section 2); they differ in width, the value embedding and the training recipe.

Exactness and cost. The kernel reproduces the token-by-token recurrence to floating-point precision: on a randomly initialised block the largest absolute difference is 4.8×10^{-7} in single precision, and the repository tests check lengths 8, 64 and 128. Per chunk it forms a 16×16 Gram matrix and solves one triangular system per head; the loop over chunks is still sequential, so a length- L forward pass takes $L/8$ chunk steps rather than L token steps. The current implementation is plain PyTorch without a fused GPU kernel.

5 The models trained so far

5.1 Data, training and evaluation

Both models are trained on enwik8 [11], the first 10^8 bytes of an English Wikipedia dump, decoded as UTF-8 and modelled at the character level. Characters that occur at least 100 times form the vocabulary (480 characters plus one id for everything else, 481 in total). The first 95% of the characters are used for training and the last 5% for evaluation. This split and vocabulary are our own; the numbers below are not comparable with published enwik8 results, which use a byte-level 90/5/5 split.

Evaluation. Each number below is measured with the model reading a window of $L \in \{128, 512, 2048, 8192\}$ characters and scoring only the *last 128* characters of the window, over 64 windows drawn at random from the evaluation text (8,192 scored characters per number). Every window *ends* at the same 64 positions, so each context length scores exactly the same characters with more or less text in front of them. (An earlier

model	bits per character, context length L			
	128	512	2048	8192
Saryu 5M (checkpoint shipped with the code)	1.671	1.586	1.586	1.586
Saryu 25M	1.535	1.433	1.433	1.433

Table 2: Character-level results on our enwik8 split, scoring identical text at every context length. Both models were trained at context 128. The columns beyond 512 repeat because the models stop using context there (Section 5.3).

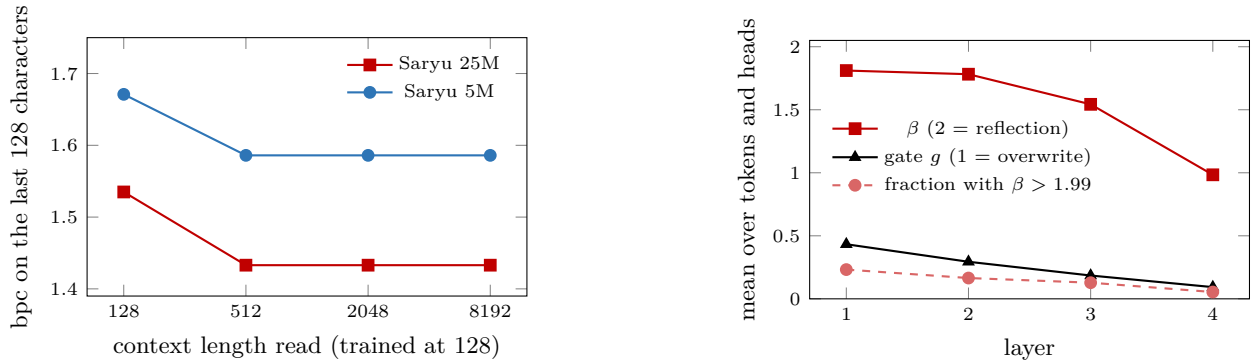


Figure 6: **Left:** bits per character against the context read, scoring identical text at every length: both models flatten at 512 characters and never improve again. **Right:** what the 25M model’s transports do on evaluation text (four windows of 1,024 characters): lower layers are close to exact reflections and rewrite often, the top layer uses softer transforms and rewrites rarely.

version of this draft drew the window starts separately for each L ; its columns scored different characters and were not comparable to each other.) The unit is bits per character (bpc); lower is better.

5.2 Results

Table 2 and Figure 6 (left) give the numbers. Three observations, stated with the caution that this is a single corpus and small models:

- **The models use about 512 characters of context and nothing before that.** Past 512 the state entering the scored text matches the long-context state to six decimals in every layer, the next-character distributions are unchanged ($\text{KL} = 0.000000$), and bpc stops moving (Section 5.3).
- **Context 128 is the only row that differs, for a mechanical reason.** At $L = 128$ the scored characters have no context at all in front of them; the KL against the long-context predictions is 4.55 at 5M and 8.39 at 25M.
- **Scale helps.** Five times the parameters and twice the training characters lower bpc by about 0.15 at every context length.

An earlier version of this draft reported $1.643 \rightarrow 1.467$ bpc from context 128 to 8192 at 25M and read it as the model using context it was never trained on. Those numbers came from windows drawn separately per length, so each column scored different text; the effect does not survive scoring identical text, and the claim is withdrawn.

5.3 How much context is used

Table 2 flattens at 512, which could mean either that the extra text carries nothing or that the model cannot see it. `evidence/effective_context.py` separates the two by comparing, at identical scored positions, the predictions and the recurrent state under context L against the longest context:

context	bpc	KL to 8192	state cosine
128	1.788326	4.553845	—
256	1.722210	0.015948	0.999525
512	1.721628	0.000012	1.000000
1024–8192	1.721638	0.000000	1.000000

(5M checkpoint, 16 windows; the 25M behaves the same, its 512-context state matching the 1024-context state to 0.999996.) Past 512 characters the state is not merely uninformative — it is identical, so the model cannot be using anything earlier. This follows from the architecture: the write is convex, so the state is a decaying mixture, and with the measured mean gate of 0.09 in the top layer, $(1 - 0.09)^{512} \approx 10^{-21}$. Reaching further needs a gate able to stay near zero for long spans, or the separate fact memory of Section 6.

5.4 What the transports learned

Figure 6 (right) summarises the 25M model’s β and gates on evaluation text; the 5M model shows the same pattern. In the first layer the mean β is 1.81 and 23% of transforms are exact reflections to within 0.01, with only 4% below $\beta = 1$; the gate averages 0.43, so the state is rewritten substantially at most tokens; 15% of first-layer gates sit at the 0.9 cap, so the cap does bind there. Moving up, both fall: in the fourth layer the mean β is 0.98, half the transforms are contractions ($\beta < 1$), and the mean gate is 0.09, so the state is kept for many tokens and edited softly. The model thus uses the full range the parameterisation allows — reflections with fast turnover low in the stack, gentle long-lived memory at the top — rather than collapsing to one regime.

6 Where Saryu goes next

Comparisons. This draft deliberately reports no comparison. The next version will compare Saryu with other architectures at matched parameters and training data — a gated recurrent network with the same block around it, matrix-state delta-rule models [21, 20, 17] and a transformer — with several seeds, a second corpus, state-tracking tasks, and throughput. The design questions above (Section 3) are only settled once those numbers exist.

A fused kernel. The chunkwise kernel is exact but written in plain PyTorch with a loop over chunks. A fused GPU kernel is needed before training at larger scale.

A memory for facts. Saryu’s state is small by design: d numbers per layer. That suits tracking *where* a sequence is, but it cannot store many independent facts, which is what associative-recall tasks measure [2]. The intended full Saryu adds a matrix memory next to the tracker (Figure 7): a delta-rule store [21] with $\beta < 1$ (no reflections inside the store, where a reflection would invert a superseded fact), addressed by content bound to the tracked state. A first version exists but does not train: because the tracker moves at every token, a fact written under one state is queried under a different one and the addresses never match. The planned fix is a tracker that changes state only when a token should change it.

7 Related work

Gated and linear recurrences. LSTMs [9] and GRUs [5] gate a vector state through nonlinear updates that must be trained sequentially. Linear attention [12] and selective state-space models [8] make the recurrence linear in the state so it can be trained in parallel, usually with diagonal transitions, which commute.

Orthogonal and Householder recurrences. Unitary and orthogonal RNNs keep the transition norm-preserving to avoid vanishing and exploding gradients [1], and Householder reflections are an efficient way to parameterise them [14]. In those models the transition is a learned constant, the same for every token.

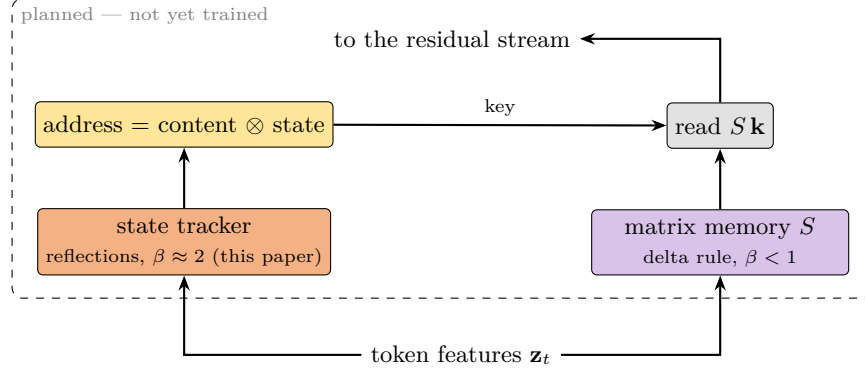


Figure 7: The intended full Saryu layer: the reflection tracker of this paper plus a matrix memory for facts, addressed by content bound to the tracked state. Not part of the trained models.

Saryu’s transforms are computed from the input at every token, and β lets each one be anything from the identity through a projection to a reflection.

Delta-rule models and state tracking. DeltaNet [21] and Gated DeltaNet [20] update a *matrix* state with generalised Householder transforms of the form $I - \beta \mathbf{k} \mathbf{k}^\top$ and write key–value outer products; extending β to $[0, 2]$ gives negative eigenvalues and state-tracking ability [7], and DeltaProduct applies several such transforms per token [17]. Saryu shares the Householder transport and the several-per-token structure. It differs in carrying a small *vector* per head rather than a matrix, in separating the write into a convex scalar-gated addition instead of a key–value outer product, in the $1 - \cos \theta$ parameterisation that makes the exact reflection reachable and stationary, and in the log-space chunk kernel that this write requires. The price is the small state discussed in Section 6.

8 Limitations of this draft

No comparison with any other model is reported. Both models are small and trained on one character-level corpus with a non-standard split; nothing here says how Saryu behaves at the scale of modern language models. Each bpc number comes from 8,192 scored characters, with different characters at each context length. The toy experiments cited in Section 3 use single transport layers and small groups. The training kernel is exact but not yet fast.

References

- [1] M. Arjovsky, A. Shah, and Y. Bengio. Unitary evolution recurrent neural networks. ICML 2016, arXiv:1511.06464, 2016.
- [2] S. Arora, S. Eyuboglu, A. Timalsina, et al. Zoology: Measuring and improving recall in efficient language models. arXiv:2312.04927, 2023.
- [3] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. arXiv:1607.06450, 2016.
- [4] C. Bischof and C. Van Loan. The WY representation for products of Householder matrices. *SIAM Journal on Scientific and Statistical Computing*, 8(1):s2–s13, 1987.
- [5] K. Cho, B. van Merriënboer, C. Gulcehre, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv:1406.1078, 2014.
- [6] S. Elfving, E. Uchibe, and K. Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. arXiv:1702.03118, 2018.

- [7] R. Grazzi, J. Siems, A. Zela, et al. Unlocking state-tracking in linear RNNs through negative eigenvalues. arXiv:2411.12537, 2024.
- [8] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. arXiv:2312.00752, 2023.
- [9] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [10] A. S. Householder. Unitary triangularization of a nonsymmetric matrix. *Journal of the ACM*, 5(4):339–342, 1958.
- [11] M. Hutter. The human knowledge compression prize. <http://prize.hutter1.net>, 2006.
- [12] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. ICML 2020, arXiv:2006.16236, 2020.
- [13] W. Merrill, J. Petty, and A. Sabharwal. The illusion of state in state-space models. arXiv:2404.08819, 2024.
- [14] Z. Mhammedi, A. Hellicar, A. Rahman, and J. Bailey. Efficient orthogonal parametrisation of recurrent neural networks using Householder reflections. ICML 2017, arXiv:1612.00188, 2017.
- [15] Y. Sarrof, Y. Veitsman, and M. Hahn. The expressive capacity of state space models: A formal language perspective. arXiv:2405.17394, 2024.
- [16] N. Shazeer. GLU variants improve transformer. arXiv:2002.05202, 2020.
- [17] J. Siems, T. Carstensen, A. Zela, et al. DeltaProduct: Improving state-tracking in linear RNNs via Householder products. arXiv:2502.10297, 2025.
- [18] L. N. Smith and N. Topin. Super-convergence: Very fast training of neural networks using large learning rates. arXiv:1708.07120, 2017.
- [19] K. Wen, Z. Li, J. Wang, et al. Understanding warmup-stable-decay learning rates: A river valley loss landscape perspective. arXiv:2410.05192, 2024.
- [20] S. Yang, J. Kautz, and A. Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. arXiv:2412.06464, 2024.
- [21] S. Yang, B. Wang, Y. Zhang, et al. Parallelizing linear transformers with the delta rule over sequence length. arXiv:2406.06484, 2024.
- [22] B. Zhang and R. Sennrich. Root mean square layer normalization. NeurIPS 2019, arXiv:1910.07467, 2019.